GROEIPROJECT
MODULE 8: DATASTRUCTUREN EN ALGORITMES

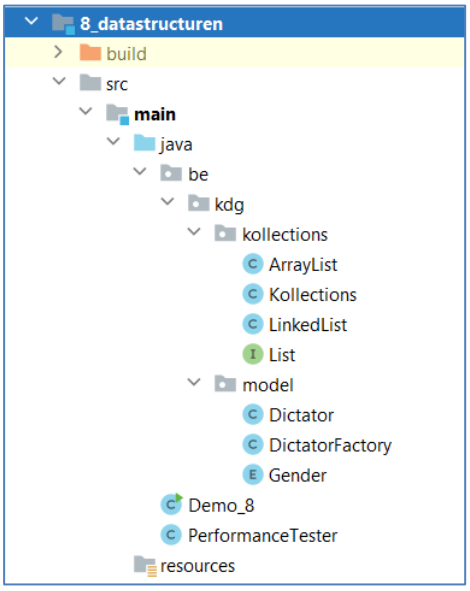
1. Voorbereiding

1.1. Open het groeiproject en maak daarin een achtste module: `m8_datastructuren`

1.2. Kopieer van module `1_herhaling` de hele model-package, behalve de multiklasse; die heb je deze week niet nodig.

1.3. In de slides van deze week gebruiken we democode die je van GitLab moet downloaden (https://gitlab.com/kdg-ti/java-2/voorbeelden/datastructures-and-algorithms-1)

In module `3_lists` is er een package `kollections`.
Kopieer heel deze package naar jouw groeiproject.
De uiteindelijke structuur (als je alle opdrachten van deze week hebt gedaan) ziet eruit zoals hiernaast →



1.4. Pas de `build.gradle` van deze module aan: verwijs naar een mainClass met de naam `Demo_8`.
Verder geen specifieke dependencies nodig.



2. Factory pattern

`2.1.` We willen dat het creëren van nieuwe basisobjecten door een Factory geregeld wordt. We gebruiken hiervoor het **Factory design pattern**, dat je beter zal leren kennen tijdens de lessen van SE2.
Een eerste aanpassing is dat het aanmaken van basisobjecten alleen door de Factory mag gebeuren, dus we gaan de constructors van de basisklasse (in ons geval: `Dictator`) afschermen, door ze **package-private** te maken.

2.2. Maak in de package `model` een nieuwe klasse die je `XxxFactory` noemt (in ons geval: `DictatorFactory`). Deze klasse zal uitsluitend static methoden bevatten voor het aanmaken van kant-en-klare basisobjecten:

- Blokkeer daarom de constructor van `XxxFactory` door hem **private** te maken.
- Voorzie een static factory-methode `newEmptyXxx` (in ons geval `newEmptyDictator`), die de default-constructor van de basisklasse oproept en dus een "leeg" basisobject retourneert.
- Voorzie een static factory-methode `newFilledXxx` (in ons geval `newFilledDictator`), die de andere constructor van de basisklasse oproept. Alle benodigde gegevens komen hier via parameters binnen en worden doorgegeven. Een "gevuld" basisobject wordt geretourneerd.
- Voorzie ook een static factory-methode `newRandomXxx` (in ons geval `newRandomDictator`), die geen parameters heeft, maar alle gegevens at random genereert en vervolgens via parameters

doorgeeft aan de constructor van de basisklasse.
Probeer deze random waarden *zo realistisch mogelijk* te genereren:

- *Datumvelden* realistisch binnen een bepaald tijdsinterval (bijvoorbeeld tussen 1900 en 2000)
- *Getalwaarden* correct binnen een bepaald bereik (bijvoorbeeld percentages tussen 0.0 en 1.0)
- *Strings* zo realistisch mogelijk: een bepaald aantal woorden, maxlengte van een woord, beginhoofdletter of niet. Maak hiervoor liefst een private hulpmethode `generateString`:

```
private static String generateString(int maxWordLength, int wordCount,
        boolean camelCase)
```

Om een realistische string samen te stellen met alfabetische karakters, hanteer je volgende simplistische regel: 1 kans op 3 voor een klinker, 2 kansen op 3 voor een medeklinker.

2.3. Gebruik de klasse `Demo_8` om daar de drie factory-methoden uit te testen. Gebruik o.a. `Stream.generate` en als Supplier jouw methode `newRandomXxx` om 30 willekeurige objecten te genereren, te sorteren en af te drukken.

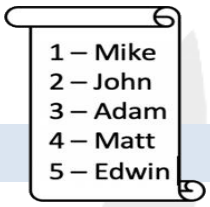
2.4. Voorbeeld van afdruk:

```
Empty dictator:
Anoniem                (°1900) Ongekend        regime: Ongekend               0,0 mln doden

Filled dictator:
Francisco Franco       (°1892) Spanje          regime: Militaire dictatuur    5,0 mln doden

30 random dictators gesorteerd op naam:
Aqxwumjgid Sucunyfobgz (°1916) Ilrieogvr       regime: idefcrz                7,9 mln doden
Azeaaefc Bao           (°1951) Cowfiouyojqqc   regime: cjnnnbdtrz             0,9 mln doden
Azreukaunfu Gllpvem    (°1906) Dqbhg           regime: shn                    2,2 mln doden
Cetnp Yialu            (°1906) Hjurkvkdcaa     regime: guriaamj              28,5 mln doden
Dhrpu Iibuti           (°1921) Vkncdfrzogskul  regime: ozgi                  20,2 mln doden
Edxccfpslnu Jhm        (°1929) Aeihwwup        regime: qcsiqzpipa            27,9 mln doden
Fciwz Utf              (°1991) Hmpvlyuudch     regime: iudoopee               6,9 mln doden

… enz…
```



3. List

3.1. Ga naar de klasse `kollections.ArrayList` en bestudeer de code die al geïmplementeerd werd. Kijk oa naar het gebruik van generics en de onderliggende array `Object[] elements`
Jouw taak: vul de methoden **`add`**, **`expand`** en **`get`** aan.

3.2. Kopieer uit de democode de volledige klasse `PerformanceTester` naar jouw groeiproject (zet ze naast de klasse `Demo_8`). Je moet in je groeiproject deze klasse nog wel aanpassen om met jouw basisklasse te werken (in ons geval dus `Dictator` ipv `Student`) .
Pas de methode `randomList` aan:

- Maak een instantie van je **eigen** ArrayList (let op dat je de juiste import doet!) en vul die in een lus met n random basisobjecten. Gebruik uiteraard je XxxFactory!
- Roep deze methode op vanuit de main in `Demo_8`; druk hier alle elementen af. (Uiteraard kan je hier geen for-each lus gebruiken!)

3.3. Ga naar de klasse `kollections.LinkedList` en bestudeer de code die al geïmplementeerd werd.
Kijk oa naar het gebruik van generics en de inner class `Node`
Jouw taak: vul de methoden **`add`** en **`get`** aan.

3.4. Test je nieuwe implementatie uit door in de methode `randomList` de `ArrayList` te vervangen door een `LinkedList`. Dat zou geen problemen mogen geven.

3.5. In de klasse `PerformanceTester` werk je de methode `compareArrayListAndLinkedList` verder uit: Meet het performantieverschil tussen de ArrayList en de LinkedList (gebruik `System.CurrentTimeMillis`):

- Voeg n keer een random object toe als eerste element (dus vooraan de lijst)
- Doe n keer een `get` van het laatste element in de lijst
- Druk telkens het gemeten resultaat af.

3.6. Roep vanuit de `main` de methode op; gebruik een voldoende grote lijst (20 000).
Mogelijke afdruk:

```
Adding 20000 to ArrayList: 882 ms.
Adding 20000 to LinkedList: 31 ms.
Getting 20000 from ArrayList: 0 ms.
Getting 20000 from LinkedList: 5695 ms.
```

3.7. Optimaliseer nu de methode `expand` in de klasse `ArrayList`. Gebruik daar geen lus, maar wel `System.arrayCopy` (die kopieert een volledig geheugenblok in één keer). Daardoor zou de performantie van het toevoegen verhoogd kunnen worden.
Controleer of dat het geval is:

```
compareArrayListAndLinkedList:
Adding 20000 to ArrayList: 111 ms.
Adding 20000 to LinkedList: 44 ms.
Getting 20000 from ArrayList: 2 ms.
Getting 20000 from LinkedList: 5450 ms.
```

4. Sorteer- en zoekalgoritmes



4.1. Ga naar de klasse `kollections.Kollections`
Maak eerst een **private default constructor**. Op die manier blokkeer je het instantiëren en kan deze klasse enkel via haar **static** methoden opgeroepen worden.

4.2. Implementeer vervolgens de static methode `selectionSort`. Bekijk in de signature van deze methode hoe er gebruik wordt gemaakt van generics en van de `Comparable` interface

4.3. Test vanuit de `main` door een List van 30 random basisobjecten te sorteren en af te drukken.

4.4. Ga naar de klasse `kollections.Kollections` en implementeer daar de methode `mergeSort`.
Deze methode is overloaded. De tweede versie is bedoeld om recursief gebruikt te worden.
De `merge` methode is al gegeven; bestudeer de code!

4.5. Test vanuit de `main` door een List van 30 random basisobjecten te sorteren en af te drukken.

4.6. Zet een **public static teller** `compareCounter` in je basisklasse en verhoog die telkens de `compareTo` wordt uitgevoerd.

4.7. In de klasse `PerformanceTester` werk je 2 methodes uit: `testSelectionSort` en `testMergeSort`: roep daar respectievelijk de methodes `selectionSort` en `mergeSort` op voor een aantal lijstgroottes (`for (n=1000;n<20000;n+=1000)`) en print telkens het aantal elementen en het aantal compares af gescheiden door puntkomma:

```
1000;499500
2000;1999000
3000;4498500
enz…
```

4.8. Roep vanuit de `main` de testmethodes op. Breng de afgedrukte waardes over in een excelsheet en vergelijk met de functies ½*n² en 3*n*log(n)
→ Dit toont aan dat `selectionSort` inderdaad O(n²) is en `mergeSort` O(n*log(n))

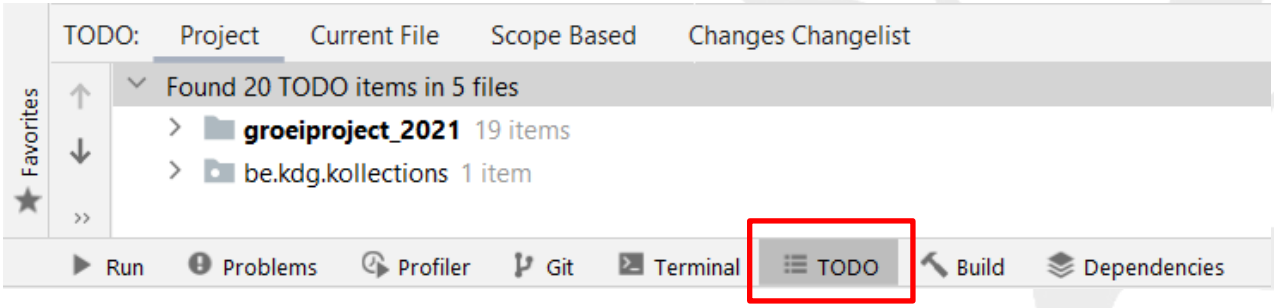
4.9. De `Kollections::quickSort` methode heb je cadeau gekregen. Test vanuit de `main` door een List van 30 random basisobjecten te sorteren en af te drukken.

4.10. Werk de implementatie van de methode `binarySearch` uit in de `Kollections` klasse. Uiteraard maak je gebruik van **recursie**!

4.11. Test uit vanuit de `main`: roep zowel `linearSearch` als `binarySearch` op met een bestaand en onbestaand element. Zorg dat de lijsten eerst gesorteerd worden! Beide methodes moeten telkens dezelfde index retourneren:

```
Index of dictator Francisco Franco: 7
Index of dictator Francisco Franco: 7
Index of dictator Anoniem: -1
Index of dictator Anoniem: -1
```

Controleer onderaan in je TODO-venster of er geen openstaande TODO's meer zijn:



OEF! Dat was een stevige boterham! Werkt alles? Proficiat! Nu nog alles pushen naar je repository op GitLab…